

Universität Stuttgart
Master of Science

Fault Tolerance

Consensus and Agreement

Marco Aiello

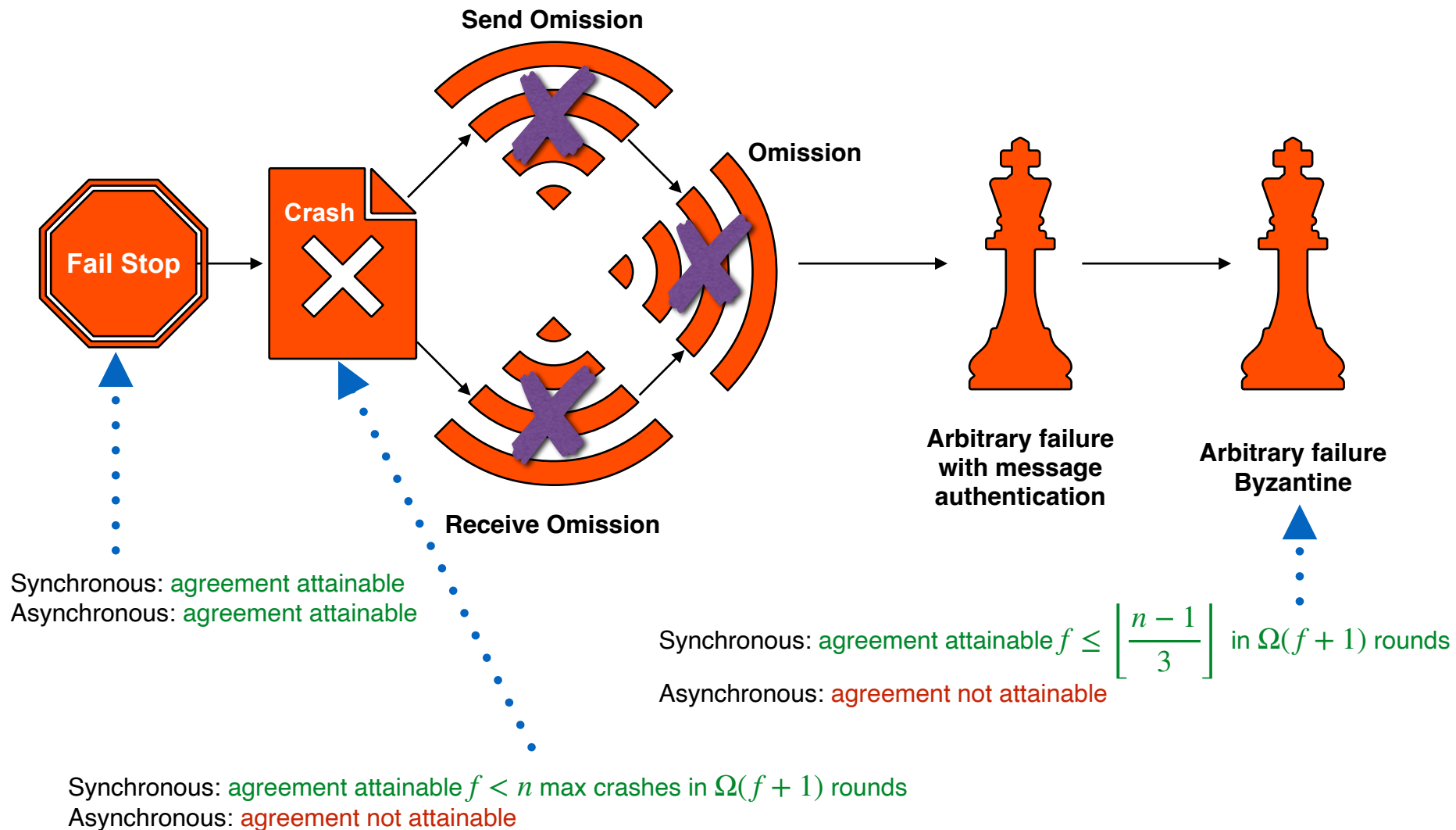
Problem Specification

- **Byzantine Agreement** (single source has an initial value):
 - Agreement: All non-faulty processes must agree on the same value.
 - Validity: If the source process is non-faulty, then the agreed upon value by all the non-faulty processes must be the same as the initial value of the source.
- **Consensus Problem** (all processes have an initial value)
 - Agreement: All non-faulty processes must agree on the same (single) value.
 - Validity: If all the non-faulty processes have the same initial value, then the agreed upon value by all the non-faulty processes must be that same value.
- **Interactive Consistency** (all processes have an initial value)
 - Agreement: All non-faulty processes must agree on the same array of values $A[v_1 \dots v_n]$.
 - Validity: If process i is non-faulty and its initial value is v_i , then all non-faulty processes agree on v_i as the i^{th} element of the array A . If the process j is faulty, then the non-faulty processes can agree on any value for $A[j]$.
- For all
 - Termination: Each non-faulty process must eventually decide on a final value.

Problem equivalence

- The three problems are *equivalent*: Having a protocol that can solve one means it can be adapted to solve the other problems
- If one has a protocol for Interactive Consistency, $A[v_1 \dots v_n]$ then it can be used for
 - Consensus $A[\bar{v}, \bar{v}, \dots \bar{v}, \bar{v},]$
 - Byzantine $A[0, 0, \dots v_i \dots 0, 0]$, where i is the source
- If one has a protocol for Byzantine Agreement, it can be used for
 - Consensus and Interactive Consistency by running it n times

Failure model



Distributed Consensus: No failures

In a (a)synchronous system with no failures, consensus is trivial.

- *Synchronous:* everybody sends their value, it arrives in a bounded time, all processes are alive and correct
- *Asynchronous:* everybody sends their value, it eventually arrives, processes could be running forever, and will eventually see the message

Impossibility of Distributed Consensus with one faulty process [Fisher, Lynch and Peterson, 1985]

- **Theorem [FLP Impossibility].** The design of a consensus protocol that tolerates failures is impossible in asynchronous distributed systems.

Impossibility holds for both shared memory and message passing systems:

- even if we assume reliable communication channels.
- even if considering only benign failures (crashes).
- even if at most one processor fails.

The problem is that in totally asynchronous systems, we cannot distinguish a dead process from a merely slow one.

FLP Proof Sketch

- In a failure-free async system, the final state is monovalent $I_0 = [0 \dots 0]$ or $I_1 = [1 \dots 1] \rightarrow$ consensus
- Ideally, if initially I_0 the decision must be 0, if I_1 must be 1
- In the face of failures, one can have a bivalent state, e.g., $I_b = [0, 0 \dots 1 \dots 1, 0]$
- A consensus algorithm should move to a monovalent state, but if there is a failure, one could have a bivalent state.
- *Proof by contradiction:* assume consensus is possible and two different starting configurations I_a and I_b . Then, take k to be the smallest value for which one transitions from 0-valent to 1-valent, and p_k crashes before taking any step. Now the system can be 0-valent or 1-valent, though p_k didn't take any step. Contradiction that consensus exists.

Distributed Consensus: Crashes

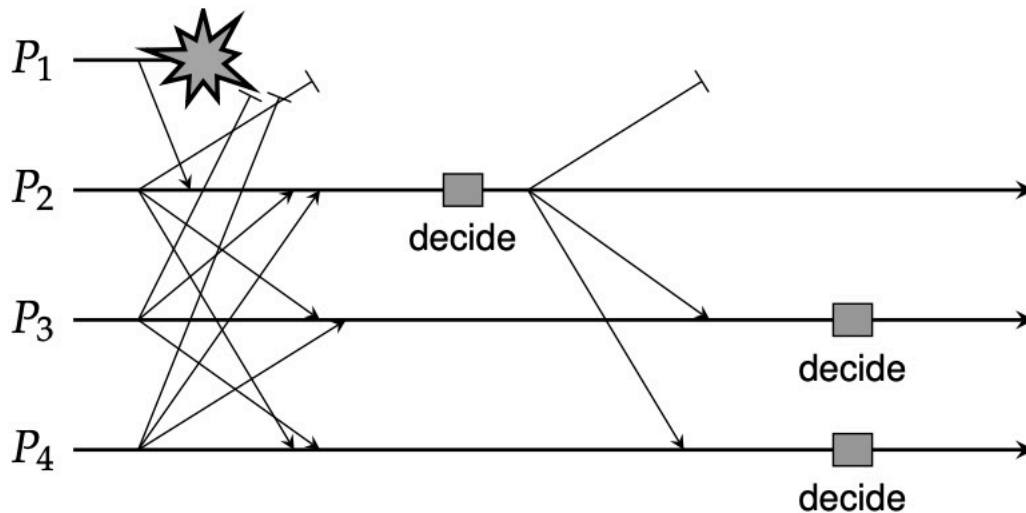
- FLP: impossible in asynchronous system
- In a synchronous system, $f < n$ failures are tolerable if complete graph in $f + 1$ rounds

```
 $V = \{x_i\}$                                 /*set  $V$  contains  $p_i$ 's input*/  
for (k:=1 to  $f+1$ )    /*round k*/  
  broadcast ( $u \in V$  :  $p_i$  has not already sent  $u$ )  
  receive set of msgs  $S_j$  from  $p_j, 1 \leq j \leq n, j \neq i$   
   $V := V \cup \bigcup_{j=1}^n S_j$     /*update  $V$  by joining it with the received sets*/  
 $y_i = \text{majority}(V)$     /*decide at  $f + 1$  round*/
```

or min function

Distributed Consensus: Crashes

- FLP: impossible in asynchronous system
- In a synchronous system, $f < n$ failures are tolerable if complete graph in $f + 1$ rounds



P_2 has received broadcast from all $\rightarrow$ decision

P_3, P_4 does not know if P_2 received from P_1 or not, cannot make decision after one round

Proof Sketch

- Suppose $\exists x : (x \in V_i) \wedge (x \notin V_j)$ with p_i, p_j non faulty
- p_i must have received x for the first time at round $f + 1$, otherwise it would have already sent it to p_j
- There is a process that sent x to p_i at round $f + 1$ that must have crashed in this round so x was not sent to p_j .
- Similarly, for the previous rounds, i.e., there is a chain of $f + 1$ distinct faulty processes that transferred x , but there are at most f faulty processes -> contradiction.

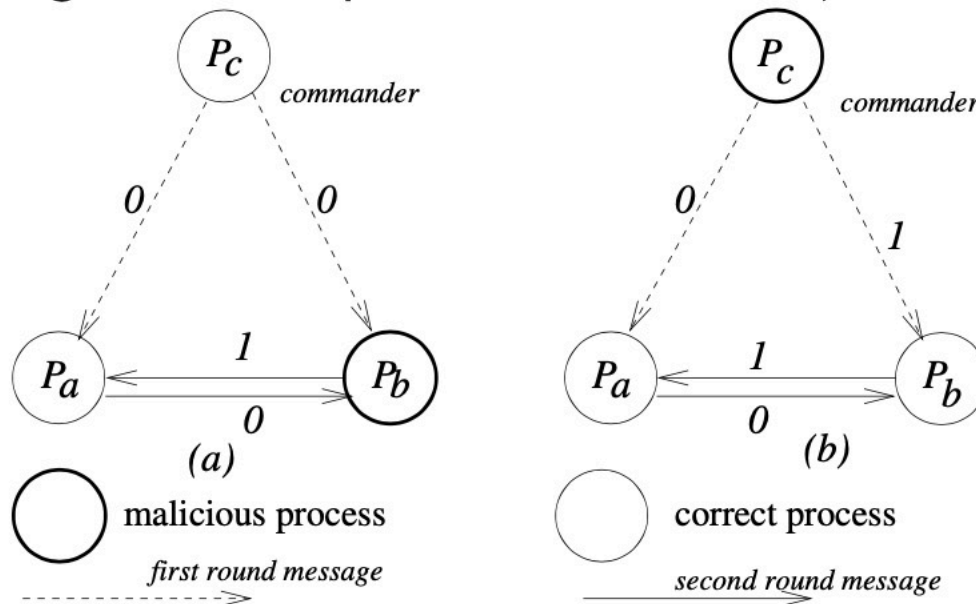
Discussion

- Message complexity $O(n^2)$
- $f + 1$ is a lower bound on the number of rounds
- Works for any $f < n$
- One can relax the complete graph requirement to $(f + 1)$ –vertex connectivity

Bizantine fault tolerance

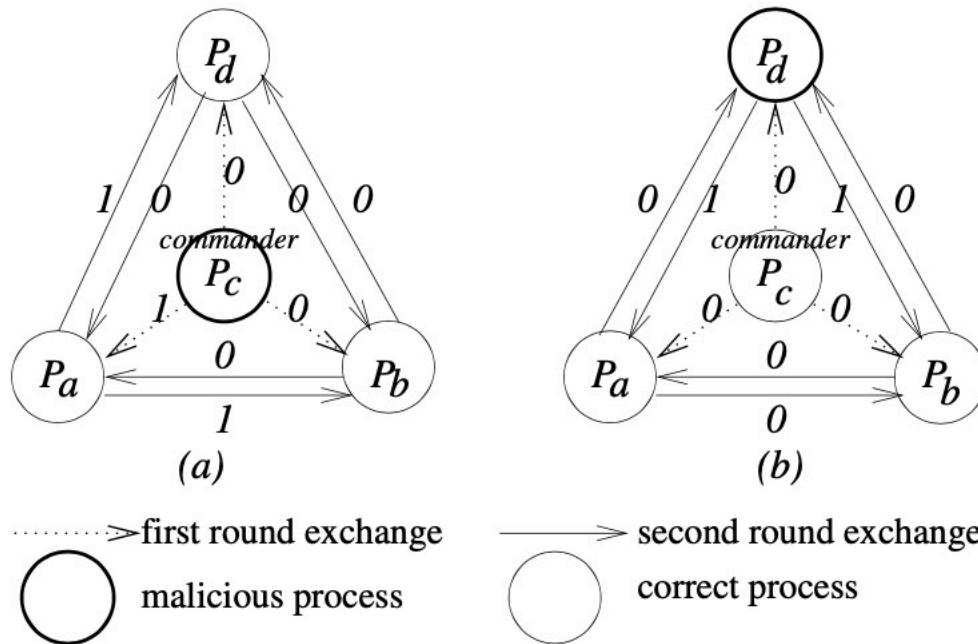
- **Theorem:** In a system with three processors, one of which is Byzantine, there is no algorithm that solves the consensus problem.

Agreement impossible when $f = 1, n = 3$.



Bizantine fault tolerance

- **Theorem:** In a system with n processors f one of which are Byzantine, there is no algorithm that solves the consensus problem if $n \leq 3f$



EiG Algorithm (recursive)

(variables)

boolean: $v \leftarrow$ initial value;

integer: $f \leftarrow$ maximum number of malicious processes, $\leq \lfloor (n - 1)/3 \rfloor$;

(message type)

$Oral_Msg(v, Dests, List, faulty)$, where

v is a boolean,

Dests is a set of destination process ids to which the message is sent,

List is a list of process ids traversed by this message, ordered from most recent to earliest,

faulty is an integer indicating the number of malicious processes to be tolerated.

$Oral_Msg(f)$, where $f > 0$:

- 1 The algorithm is initiated by the Commander, who sends his source value v to all other processes using a $OM(v, N, \langle i \rangle, f)$ message. The commander returns his own value v and terminates.
- 2 **[Recursion unfolding:]** For each message of the form $OM(v_j, Dests, List, f')$ received in this round from some process j , the process i uses the value v_j it receives from the source, and using that value, acts as a *new* source. (If no value is received, a default value is assumed.)
To act as a new source, the process i initiates $Oral_Msg(f' - 1)$, wherein it sends $OM(v_j, Dests - \{i\}, concat(\langle i \rangle, L), (f' - 1))$ to destinations not in $concat(\langle i \rangle, L)$ in the next round.
- 3 **[Recursion folding:]** For each message of the form $OM(v_j, Dests, List, f')$ received in Step 2, each process i has computed the agreement value v_k , for each k not in $List$ and $k \neq i$, corresponding to the value received from P_k after traversing the nodes in $List$, at one level lower in the recursion. If it receives no value in this round, it uses a default value. Process i then uses the value $majority_{k \notin List, k \neq i}(v_j, v_k)$ as the agreement value and returns it to the next higher level in the recursive invocation.

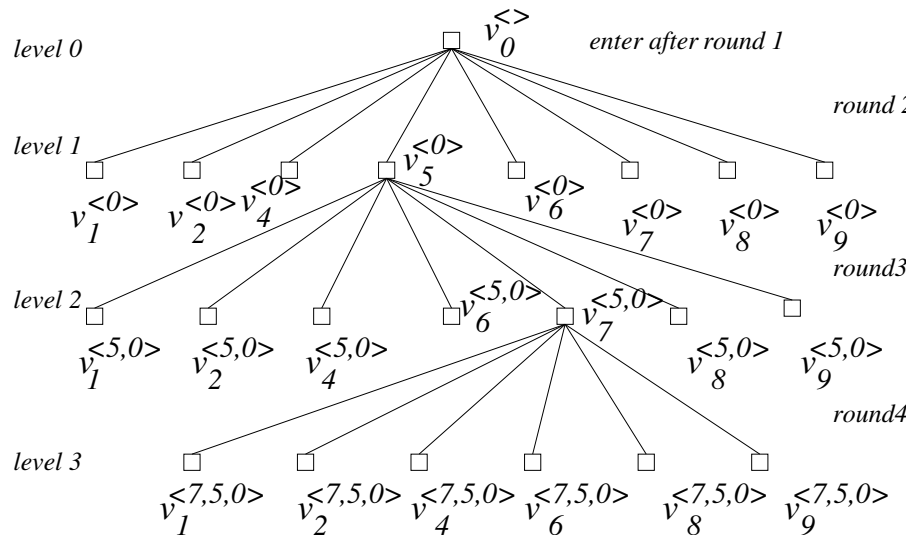
Oral_Msg(0):

- 1 **[Recursion unfolding:]** Process acts as a source and sends its value to each other process.
- 2 **[Recursion folding:]** Each process uses the value it receives from the other sources, and uses that value as the agreement value. If no value is received, a default value is assumed.

Exponential Information Gathering Algorithm

round number	a message has already visited	aims to tolerate these many failures	and each message gets sent to	total number of messages in round
1	1	f	$n - 1$	$n - 1$
2	2	$f - 1$	$n - 2$	$(n - 1) \cdot (n - 2)$
...	...	...	...	...
x	x	$(f + 1) - x$	$n - x$	$(n - 1)(n - 2) \dots (n - x)$
$x + 1$	$x + 1$	$(f + 1) - x - 1$	$n - x - 1$	$(n - 1)(n - 2) \dots (n - x - 1)$
$f + 1$	$f + 1$	0	$n - f - 1$	$(n - 1)(n - 2) \dots (n - f - 1)$

Example



Some branches of the tree at P_3 . In

this example, $n = 10$, $f = 3$, commander is P_0 .

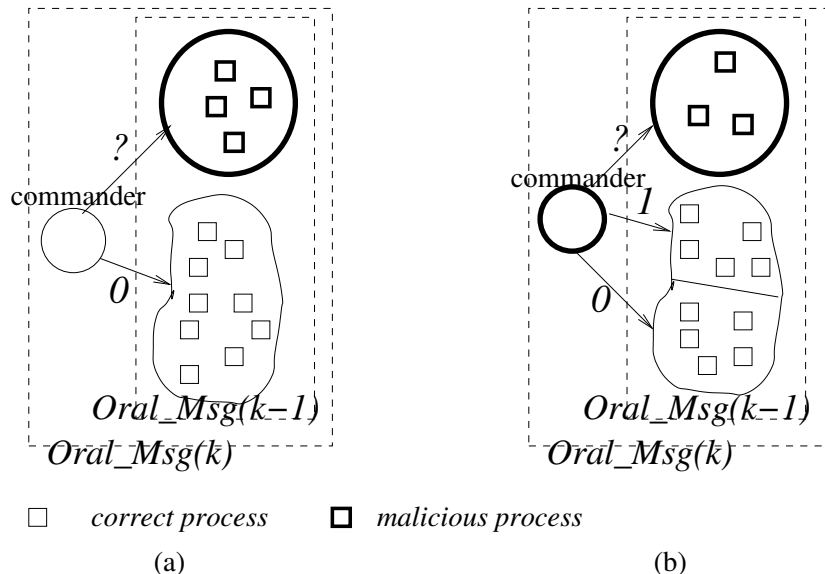
- (round 1) P_0 sends its value to all other processes using $Oral_Msg(3)$, including to P_3 .
- (round 2) P_3 sends 8 messages to others (excl. P_0 and P_3) using $Oral_Msg(2)$. P_3 also receives 8 messages.
- (round 3) P_3 sends $8 \times 7 = 56$ messages to all others using $Oral_Msg(1)$; P_3 also receives 56 messages.
- (round 4) P_3 sends $56 \times 6 = 336$ messages to all others using $Oral_Msg(0)$; P_3 also receives 336 messages. The received values are used as estimates of the majority function at this level of recursion.

Example

An example of the majority computation is as follows.

- P_3 revises its estimate of $v_7^{\langle 5,0 \rangle}$ by taking $\text{majority}(v_7^{\langle 5,0 \rangle}, v_1^{\langle 7,5,0 \rangle}, v_2^{\langle 7,5,0 \rangle}, v_4^{\langle 7,5,0 \rangle}, v_6^{\langle 7,5,0 \rangle}, v_8^{\langle 7,5,0 \rangle}, v_9^{\langle 7,5,0 \rangle})$. Similarly for the other nodes at level 2 of the tree.
- P_3 revises its estimate of $v_5^{\langle 0 \rangle}$ by taking $\text{majority}(v_5^{\langle 0 \rangle}, v_1^{\langle 5,0 \rangle}, v_2^{\langle 5,0 \rangle}, v_4^{\langle 5,0 \rangle}, v_6^{\langle 5,0 \rangle}, v_7^{\langle 5,0 \rangle}, v_8^{\langle 5,0 \rangle}, v_9^{\langle 5,0 \rangle})$. Similarly for the other nodes at level 1 of the tree.
- P_3 revises its estimate of $v_0^{\langle \rangle}$ by taking $\text{majority}(v_0^{\langle \rangle}, v_1^{\langle 0 \rangle}, v_2^{\langle 0 \rangle}, v_4^{\langle 0 \rangle}, v_5^{\langle 0 \rangle}, v_6^{\langle 0 \rangle}, v_7^{\langle 0 \rangle}, v_8^{\langle 0 \rangle}, v_9^{\langle 0 \rangle})$. This is the consensus value.

Impact of byzantine processes



The effects of a loyal or a disloyal commander in a system with $n = 14$ and $f = 4$. The subsystems that need to tolerate k and $k - 1$ traitors are shown for two cases. (a) Loyal commander. (b) No assumptions about commander.

(a) the commander who invokes $Oral_Msg(x)$ is loyal, so all the loyal processes have the same estimate. Although the subsystem of $3x$ processes has x malicious processes, all the loyal processes have the same view to begin with. Even if this case repeats for each nested invocation of $Oral_Msg$, even after x rounds, among the processes, the loyal processes are in a simple majority, so the majority function works in having them maintain the same common view of the loyal commander's value.

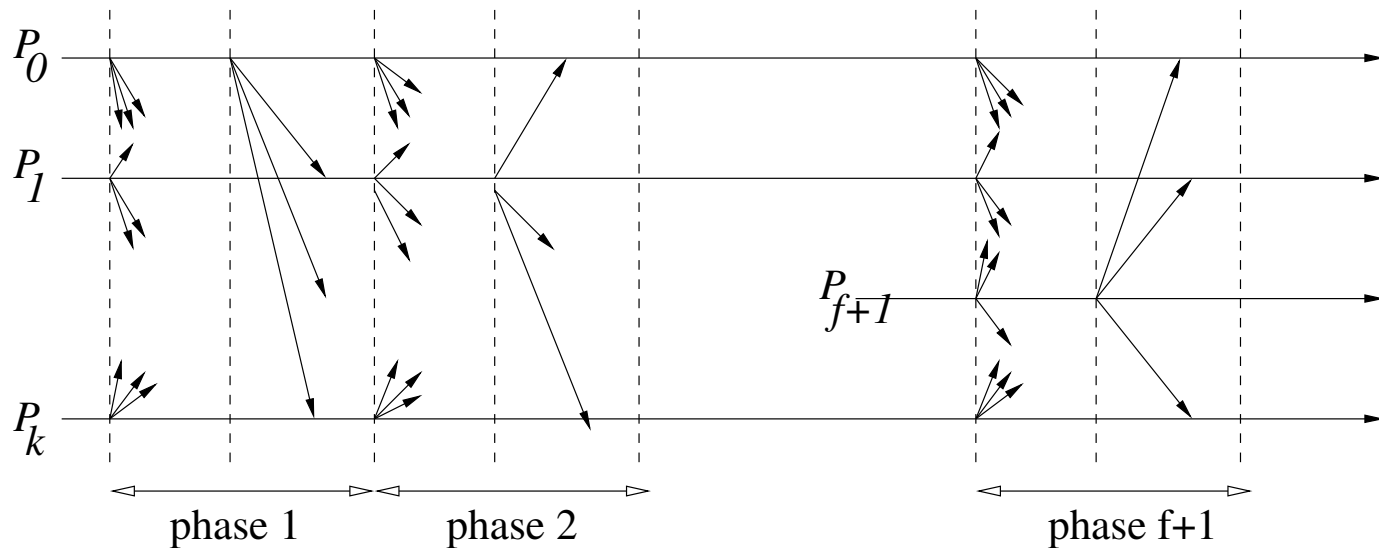
(b) the commander who invokes $Oral_Msg(x)$ may be malicious and can send conflicting values to the loyal processes. The subsystem of $3x$ processes has $x - 1$ malicious processes, but all the loyal processes do not have the same view to begin with.

Discussion

- Message complexity: $n^2(f + 1)$ of exponential size, sub-optimal
- Number of rounds: $f + 1$ optimal
- Fault tolerance: $f \leq \left\lfloor \frac{n - 1}{3} \right\rfloor$ optimal
- One can relax the complete graph requirement to $2f + 1$ connected graph

Phase King Algorithm [Berman-Garray-Perry1989]

- Improvement on message complexity at the cost of lower tolerance
- Messages of constant size, but $2(f + 1)$ rounds and with $n > 4f$



Phase King Algorithm

(variables)

boolean: $v \leftarrow$ initial value;

integer: $f \leftarrow$ maximum number of malicious processes, $f < \lceil n/4 \rceil$;

(1) Each process executes the following $f + 1$ phases, where $f < n/4$:

(1a) **for** $phase = 1$ **to** $f + 1$ **do**

(1b) Execute the following Round 1 actions: // actions in round one of each phase

(1c) **broadcast** v to all processes;

(1d) **await** value v_j from each process P_j ;

(1e) $majority \leftarrow$ the value among the v_j that occurs $> n/2$ times (default if no maj.);

(1f) $mult \leftarrow$ number of times that $majority$ occurs;

(1g) Execute the following Round 2 actions: // actions in round two of each phase

(1h) **if** $i = phase$ **then** // only the phase leader executes this send step

(1i) **broadcast** $majority$ to all processes;

(1j) **receive** $tiebreaker$ from P_{phase} (default value if nothing is received);

(1k) **if** $mult > n/2 + f$ **then**

(1l) $v \leftarrow majority$;

(1m) **else** $v \leftarrow tiebreaker$;

(1n) **if** $phase = f + 1$ **then**

(1o) output decision value v .

Proof Sketch

- Among all the rounds at least one has a Phase King who is correct
- If all non-malicious processes have the value x at the start of a round, they will continue to have x as the consensus value at the end of the round. Since
$$n > 4f \implies \frac{n}{2} > 2f \implies n > \frac{n}{2} + 2f \implies n - f > \frac{n}{2} + f$$
- If two process have already more than $\frac{n}{2} + f$ majority, they are stable and correct
- If one uses the majority and one tiebreaker, one stable, one will become correct when the Phase King is non faulty
- If both use tiebreaker, they will become correct when the Phase King is non faulty and they will stay that way

$$\frac{n}{2} + f = 3.5 \implies \text{threshold is } 4$$

Example with 5 Processes

	Initial value	Phase 1 Round 1	mult	Round 2 (king)	Phase 2 Round 1	mult
1	0	00110	3	0	00000	5
2	0	00111	3	0	00001	4
3	1	00110	3	0	00000	5
4	1	00111	3	0	00001	4
5	Byzantine					

$$\frac{n}{2} + f = 3.5 \implies \text{threshold is 4}$$

Example with 5 Processes

	Initial value	Phase 1 Round 1	mult	Round 2 (king)	Phase 2 Round 1	mult	Phase 2 Round 1	mult
1	Byzantine							
2	0	00011	3	0	00011	3	0	4
3	0	10011	3	0	10011	3	0	4
4	1	00011	3	1	00011	3	0	4
5	1	10011	3	1	10011	3	0	4

Recap

